Experiment No . 5

AIM :

To study the functioning of Arrays and Loops in JavaScript:

Part a: Create and manipulate arrays using push(), pop(), shift(), unshift(), splice(), and slice().

Part b: Use iteration and transformation array methods: map(), filter(), reduce(), and forEach().

SOFTWARE / TOOLS REQUIRED :

- Any text/code editor (e.g., Visual Studio Code, Sublime Text)
- Modern Web Browser (e.g., Google Chrome, Microsoft Edge, Mozilla Firefox) to render HTML and execute JavaScript via Console
- Basic knowledge of HTML5 and modern JavaScript (ES6+)

THEORY :

1. **JavaScript Arrays:**
   Arrays are high-level, list-like objects used to store multiple values in a single variable. They facilitate the efficient management, traversal, and dynamic manipulation of data collections using built-in prototype methods.
2. **Array Methods & Functional Iteration:**
   JavaScript provides high-order functions such as `map()`, `filter()`, `reduce()`, and `forEach()` to operate on array elements declaratively. These functional programming utilities simplify complex tasks like transforming arrays, filtering elements based on logical predicates, and accumulating values.
3. **JavaScript Objects:**
   Objects store data in key-value pairs and represent real-world entities. Arrays in JavaScript are specialized objects with numerical keys (indices) and a dynamic auto-updating `length` property.

A. Array Creation & Manipulation Methods

Method	Type / Return	Description
`push(...items)`	Mutates (Returns new length)	Adds one or more elements to the end of an array.
`pop()`	Mutates (Returns removed element)	Removes and returns the last element from an array.
`shift()`	Mutates (Returns removed element)	Removes and returns the first element from an array.
`unshift(...items)`	Mutates (Returns new length)	Adds one or more elements to the beginning of an array.
`splice(start, count, ...items)`	Mutates (Returns deleted items)	Adds, removes, or replaces elements at a specific index.
`slice(start, end)`	Non-Mutating (Returns new array)	Extracts a shallow copy of a portion of an array.

B. Iteration & Transformation Methods

Method	Type / Return	Description
`forEach(callback)`	Iteration (Returns undefined)	Executes a provided function once for each array element.
`map(callback)`	Transformation (Returns new array)	Creates a new array populated with the results of calling a provided function on every element.
`filter(callback)`	Selection (Returns new array)	Creates a new array with all elements that pass the predicate test function.
`reduce(callback, initialVal)`	Reduction (Returns accumulated value)	Executes a reducer function on each element, accumulating results into a single output value.

AIM OF THE CASE STUDY :

Define an array of numerical values (e.g., `let numbers = [12, 45, 2, 89, 34];`) and write JavaScript logic to evaluate the elements and determine both the maximum (highest) and minimum (lowest) values within the collection.



OUTPUT :

If given the input array [12, 45, 2, 89, 34], the program execution yields:

- **Maximum value:** 89
- **Minimum value:** 2

CONCLUSION :

By completing this experiment, we have successfully studied and implemented JavaScript array operations, dynamic manipulation functions (`push, pop, shift, unshift, splice, slice`), and functional iteration utilities (`map, filter, reduce, forEach`).

Furthermore, through the case study, we demonstrated how to store numerical data within an array data structure and programmatically search, evaluate, and extract key statistical boundary limits (minimum and maximum values) using modern ES6+ syntax and built-in JavaScript methods.